

Q) to Create a dataframe using dictionary which includes student name, semester, marks, height, Age, study hours. Also include some null value.

(i) Value count of student in sems

(ii) Outliers in marks column.

(iii) Correlation in marks & study hours

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
data = {
    "name": ["A", "B", "C", "D", "E"],
```

```
    "semester": [2, 2, 1, 1, 1],
```

```
    "Marks": [100, 100, 99, 97, 95],
```

```
    "age": [18, 17, 18, np.nan, np.nan],
```

```
    "height": [162, 165, 177, 180, 170],
```

```
    "study hours": [8, 9, 7, np.nan, np.nan]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
print(df[["semester"]].value_counts())
```

```
q1 = np.percentile([90, 100], 25)
```

```
q3 = np.percentile([90, 100], 75)
```

```
iqr = q3 - q1
```


lower = $[(-1.5) * iq_8] + q_1$

upper = $[(1.5) * iq_8] + q_3$

print(lower)

print(upper)

marks_study hours = df["marks"].corr(df["study hours"])

print("correlation is :", marks_study hours)

n = [100, 100, 99, 97, 95]

y = [8, 9, 4, np.nan, np.nan]

plt.plot(n, y)

plt.show()

Output -

	Student	Semester	marks	Age	Height	StudyHrs.
0	Lisa	1	78.0	17.0	142	5.0
1	Lexi	1	88.0	18.0	158	16.0
2	Jennie	3	NaN	19.0	160	7.0
3						
4						
5						
6						
7						
8						
9						
10						
11						
12	Aries	5	70.0	20.0	174	10.0

Dataframe after filling null values.

	Student	Semester	marks	Age	Height	StudyHrs.
0	Lisa	1	78.0	17.0	142	5.0
1	Lexi	1	88.0	18.0	158	16.0
2	Jennie	2	89.0	19.0	160	7.0
3						
4						
5						
6						
7						
8						
9						
10						
11						
12	Aries	5	70.0	10.0	174	10.0

Q) There are 2 dataset given in which:
 → one dataset contains medical records
 → second dataset contains patient records.

Now preprocess together.

import pandas as pd

df1 = pd.read_csv("C:\\users\\GUE\\Downloads\\medical-records.csv")

print(df1)

df1.head(10)

df1.tail(10)

df1.info()

df1.notnull()

df2 = pd.read_csv("C:\\users\\GUE\\Downloads\\patient-details.csv")

print(df2)

df2.head(10)

df2.tail(10)

df2.info()

df2.notnull()

df2.duplicated()

merge-right = pd.merge(df1, df2, how="right")

merge-right

merge-left = pd.merge(df1, df2, how="left")

merge-left

merge-outer = pd.merge(df1, df2, how="outer")

merge-outer

merge-inner = pd.merge(df1, df2, how="inner")

merge-inner

print(df1.describe())

print(df2.describe())

print(df1.isnull().sum().sum())

print(df2.isnull().sum().sum())


```

print(df1.duplicated())
print(df2.duplicated())

hyper_pat = df1[df1['diagnosis'] == "hypertension"]
print(hyper_pat)

df2 = df2.replace()

```

Q3) Write the code of DFS by the reference of graph.

```
def dfs(start, goal, graph vis = None):
```

```
    if vis is None:
```

```
        vis = set()
```

```
    print(start)
```

```
    if start == goal:
```

```
        return True
```

```
    vis.add(start)
```

```
    for v in graph.get(start, []):
```

```
        if v not in vis:
```

```
            if dfs(v, goal, graph, vis):
```

```
                return True
```

```
    return False
```

```
graph = {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['A', 'C', 'D'],
```

```
    'C': ['A', 'B', 'E'],
```

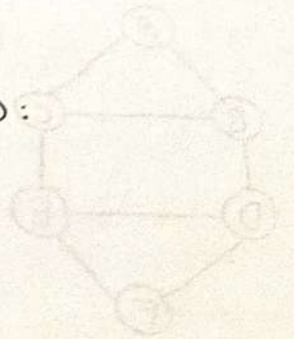
```
    'D': ['B', 'E', 'F'],
```

```
    'E': ['C', 'D', 'F'],
```

```
    'F': ['D', 'E']
```

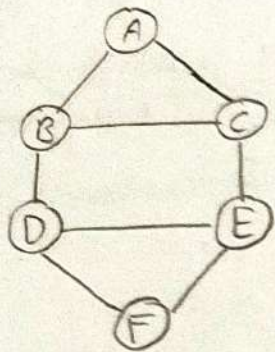
```
}
```

```
print(dfs('A', 'F', graph))
```



03/02/2026

Q3)



A-B
A-C
B-D
C-E
D-F
D-E

• Denote -
↳ parent → U
↳ child → V

Write code for DFS for the given graph.

```

Ans. def dfs(start, goal, graph, vis = None):
    if vis is None:
        vis = set()
    print(start)
    if start == goal:
        return True
    vis.add(start)
    for child in graph[start]:
        if child not in vis:
            if dfs(child, goal, graph, vis):
                return True
    graph = {}
    n = int(input("Enter no. of nodes = "))
    for i in range(n):
        k = input("enter node: ")
        graph[k] = []
        m = int(input("how many neighbours: "))
        for i in range(m):
            v = input("enter neighbours: ")
            graph[k].append(v)
  
```

graph

S = input("enter starting point: ")
E = input("enter ending point: ")
print(dfs(S, E, graph))

Output

Enter no. of nodes = 6

enter node: A

how many neighbours: 2

enter neighbour: B

enter neighbour: C

enter node: B

how many neighbours: 3

enter neighbour: A

enter neighbour: B

enter neighbour: E

enter node: D

how many neighbours: 3

enter neighbour: B

enter neighbour: E

enter neighbour: F

enter node: E

how many neighbours: 3

enter neighbour: C

enter neighbour: D

enter neighbour: F

enter node: F

how many neighbours: 2

enter neighbour: D

enter neighbour: E

enter starting point: A

enter ending point: F

A
B
C
D
E
F
True

Code for A star

```
import network as nx
graph = nx.Graph()
graph
```

while True:

```
choice = int(input("Insert nodes & centre edge! press 1 for Yes & 2 for no"))
```

if choice == 1:

```
node1 = input("enter node1").strip().upper()
```

```
node2 = input("enter node2").strip().upper()
```

```
weight = int(input("enter weight"))
```

```
graph.add_edge(node1, node2, weight=weight)
```

else:

```
break
```

Q) What is a graph? Why do we require a graph?

Q) Explain Types of graph

Q) What is DFS? Why do we use it? 5 real life applications & how it is used.
(Depth First Search)

Q) What is BFS? Why do we use it? Give 5 real life applications & how it is used.
(Breadth First search)

Q) Differentiate BFS & DFS.

Q) Why are we using normalisation?

Q) Create a dataframe with name, age, marks column & analysis the dataframe. Draw boxplot of age.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

data = {'name': ['priya', 'riya', 'arvi', 'devansh'],
        'age': [18, 19, 20, 19],
        'marks': [80, 88, 90, 91]}
```

```
df = pd.DataFrame(data)
print(df)
print(df.shape)
print(df.size)
df['name'] = df['name'].fillna(0)
plt.figure(figsize=(8,5))
plt.boxplot(df['age'])
plt.title('Boxplot to find outliers')
plt.ylabel('Values')
plt.show()
```

Output

Cleaned Dataframe:

	Student	Marks	Age	Sub	
0	Drysan	89.0	20.0	Phy	Student 0
1	Lexi	67.0	19.0	Chem	Marks 0
2	Luna				Age 0
...	...	...	...	...	Subject 0
10	Jay	90.0	21.0	AIM	

dtype: int 64

Shape of data frame: (11, 4)
Missing value count: 0

Q3) Medical Record Column:

	record-id	Patient-id	diagnosis	BP	sugar	visit cost
0	R101	P013	diabetes	120/80	140	2000
1	R102	P106	Hypertension	140/80	140	2700
2	R103	P088	diabetes	Low	141	5000
...	...	...	...	...	...	...
120	R220	P015	diabetic	Low	180	1500

Patient Details Column:-

	Patient-id	name	age	gender	city	High
0	P001	Raksha	25	F	Delhi	High
1	P002	trysan	60	F	Mumbai	110/80
2	P003	Nena	45	M	Kolkata	Low
3	P004	Dasi	60	F	Gurgaon	Low
...	...	...	...	...	...	...
120	P010	Drysan	60	Female	Delhi	High

Total visits of every diagnosis:

Diagnosis:

Asthma	78500
Diabetes	95500
Hypertension	100500

Total patients in each diagnosis:

Diagnosis

Asthma	40
Diabetes	43
Hypertension	42

Analysis & merging of 2 csv files-

```
① import pandas as pd
import numpy as np
df = pd.read_csv("patient to csv")
print(df.head())
print(df.info())
df['sugar-level'] = df['sugar-level'].fillna(df['sugar-level'].mean())
df['bp'] = df['bp'].replace(['high', '1.5'], [140/190, 120/80])
print(df.to_string(index=False))
sort_df = df.groupby('diagnosis')['visit-cost'].apply(max)
print(sort_df)
```

```
② df1 = pd.read_csv('patient2.csv')
df1['gender'] = df1['gender'].replace(['female', 'Male'], ['F', 'M'])
df1['height'] = df1['height'].replace('abc', '185')
df1['name'] = df1['name'].fillna("unknown")
df1['age'] = df1['age'].fillna(df1['age'].mean())
print(df1.to_string(index=False))
```

```
③ df2 = pd.merge(df, df1, on='patient-id', how='inner')
print(df2.to_string(index=False))
```

Create a dataset for medical which has columns such as patient ID, name, age, diagnosis, sugar levels, BP & income of the patient.

15

Give are doing normalisation to match the similar kinds of values, bringing them in a particular range.
→ for easy analysis → for detailed study.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'Patient-ID': ['101', '102', '103', '100', '105', '106', '107', '108', '109', '110',
                  '111', '112', '113', '114', '115'],
    'Name': ['Max', 'Drew', 'Dryson', 'Tyson', 'Ksifer', 'Luna', 'Lexi',
            'Riviera', 'Josh', 'Britann', 'Brister', 'Sun Jae', 'Peter',
            'Kim', 'Jennifer'],
    'diagnosis': ['hypertension', 'Asthma', 'diabetes', 'Myopia', 'nightblindness', 'Covid-19', 'hypertension', 'depression', 'anxiety',
                 'Myopia', 'Asthma', 'diabetes', 'depression', 'Asthma', 'Asthma'],
    'Age': [
        180, 110, 130, 140, 150, 230, 240, 250, 150, 120, 130, 170,
        150, 150, 140],
    'income': [np.nan, 500000, 40000, 50000, 45000, 20000, 47000,
               80000, 14000, 15000, 20000, 70000, 82000, 72000,
               37000]
```

```
df = pd.DataFrame(data)
print(df)
```


Q) What is graph & why do we use it?

A graph is a non-linear data structure consisting of

- Vertices (Nodes) - represents objects
- Edges (Links) - represents relationship b/w objects

Mathematical Representation - $G = (V, E)$

where V = set of vertices, E = set of Edges

Why we use graphs -

- 1) To represent real-world networks
- 2) To model relationship b/w data
- 3) Used in social network
- 4) Used in network routing & AI problems
- 5) Used in path finding & navigation

Types of graph -

- (1) Cyclic graph: contains at least one cycle

You can start at a node & return to it

Eg: Road loop

- (2) Acyclic Graph - no cycles present

If directed - called Directed Acyclic graph (DAG)

Eg - Task scheduling

- (3) Connected Graph: Every vertex is reachable from every other vertex. At least one path exists b/w any 2 nodes.

Eg - Fully connected computer network.

- (4) Complete graph - Every vertex is connected to every other vertex.

$$\text{no. of edges} = \frac{n(n-1)}{2}$$

Eg - Fully connected LAN

- (5) Disconnected Graph - Some vertices are not connected.

Contains two / more separate components

Eg - separate special groups.

- (6) Bipartite Graph: Vertices divided into 2 sets. Edges connect vertices from different set only.

Eg - Job applicants & companies matching.

- (7) Undirected Graph - Edges have no direction

Connection is two way

Eg - Friendship network

- (8) Directed Graph - Edges have directions

Connection is one way

Eg - Instagram follow system

Q) What is Dfs & why do we use it?

Depth First Search (dfs) is a graph transversal algorithm used to visit all nodes of a graph or tree. It starts from a source nodes & explores as far as possible along each branch before backtracking.

Working Principle:

- Starts from a root / starting node.
- visit an adjacent unvisited node.
- Continue visiting deeper nodes.
- If no unvisited is left → Backtrack
- Repeat until all nodes are visited.

DFS is implemented using → STACK (LIFO) or Recursion

class

- 1) Helps find a path b/w nodes.
- 2) Detects loops in graphs.
- 3) Used in task scheduling - Topological Sorting
- 4) Finds exist path.
- 5) Checks connectivity.

5 - Real world examples -

- 1) File system searching - Used to search files & folders in a computer directory. It explores one folder in a computer directory. It explores one folder completely before moving to another.
- 2) Cycle Detection in Network - DFS helps detect loop in computer networks or circuits.
- 3) Web Crawling: Search engines use DFS-like techniques to explore web pages by following links deeply before moving to other links.
- 4) Maze solving - DFS is used to explore all possible paths in a maze by going deep into one path until the exit or dead end is found.
- 5) Social Network Analysis - Used to find connected graphs or communities by exploring one connection chain deeply.

Q) What is bfs & why do we use it?

Breadth First Search (BFS) is a graph traversal algorithm that visits node level by level. It first visits all neighbouring nodes, then moves to next level neighbours.

Working Principle:

- 1) Start from a source node
- 2) Visit all adjacent node first
- 3) Then visit neighbours of those nodes.
- 4) Continue until all nodes are visited.

~~DFS~~ BFS $\rightarrow$ is implemented $\rightarrow$ Queue $\rightarrow$ (PFFD)

Why use it -

- 1) To find ~~the~~ shortest path in unweighted graph.
- 2) Used in AI search problem.
- 3) Helps in peer to peer networks.
- 4) Used in network broadcasting.

5) Real Life Example -

- 1) Network Broadcasting - Sending data packets to all connected system level by level.
- 2) Load Media - Friend suggestions $\rightarrow$ Find friends at level 1 $\rightarrow$ level 2 $\rightarrow$ level 3 connections.
- 3) Web Crawlers $\rightarrow$ Search engines scan pages layer by layer from main page.
- 4) GPS Navigation $\rightarrow$ Used to find shortest route when distance weight is equal.
- 5) Finding Nearest Devices $\rightarrow$ Like nearest hospital, petrol pump, or ATM from location.

Differentiate b/w BFS & DFS -

Basis.	BFS	DFS
Traversal Style	Level by Level	Goes deep into branches first.
Data Structure Used.	Queue	Stack
Shortest Path	Finds shortest path in unweighted graph	Does not guarantee shortest path
Memory Usage	Higher	Lower
Speed	Slower for deep search	Faster
Applications	Shortest Path	Cycle detection

Q) What is normalisation & why do we use it.

Normalisation is a data preprocessing technique used to scale numerical data into a fixed range (usually 0 to 1)

Common Formula (Min - Max Normalisation)

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Uses -

- 1) Removes scale differences b/w features -
- 2) Improves machine learning model performances.
- 3) Prevents one feature dominating others.
- 4) Required for distance based algorithms.

Where is it used -

- Machine learning, Neural Networks, K-NN Algorithm, Image Processing, Data Mining

Q) Write a A* algorithm code & make a graph & calculate the path.

→ import networks as nx
graph = nx.Graph()
while True:

ch = int(input('want to insert node & edges (press 1 for yes & 0 for no:)'))

if ch == 1:

node1 = input("enter node 1 ").strip().upper()

node2 = input("enter node 2 ").strip().upper()

node3 = input("enter node 3 ").strip().upper()

wt = int(input("enter weight b/w node 1 & 2 "))
graph.add_edge(node1, node2, weight=wt)

else:

break

heuristic = {}

nodes = list(graph.nodes())

print('enter heuristics value for each other')

for node in nodes:

heuristic[node] = int(input(f'h {node} = '))

def h(n1, n2):

return heuristic[n1]

st = input("start node ").strip().upper()

go = input("goal node ").strip().upper()

path = nx.astar_path(graph, st, heuristic=h, weight='weight')

print("a* path: ", path)

Output -

Want to insert & enter edges (press 1 for yes & 2 for no):

Enter node 1: a

Enter node 2: b

Enter the weight b/w node 1 & node 2: 4

Want to insert & enter edges (press 1 for yes & 2 for no): 1

Enter node 1: b

Enter node 2: c

Enter the weight b/w node 1 & node 2: 5

Want to insert & enter edges (press 1 for yes & 2 for No): 1

Enter node 1: b

Enter node 2: c

Enter the weight b/w node 1 & node 2: 6

Want to insert & enter edges (press 1 for yes & 2 for no): 2

Enter heuristic values for each node:

Enter h(A): 3

Enter h(B): 5

Enter h(C): 2

Start node: a

Goal node: c

A* path: ['A', 'C']

Q) Multi-Linear Regression - House Price Prediction

(i) Dataset 1 - Area, length, breadth, Rooms

2 → 2BHK > Big apartment } Classification model
2BHK < Small apartment }

(ii) import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = { "area": [1000, 1200, 1500, 1800, 2000, 2200, 2500, 2700],

"length": [20, 24, 25, 30, 32, 35, 40, 45],

"breadth": [50, 50, 60, 60, 62, 63, 62, 60],

"rooms": [2, 3, 3, 4, 5, 5, 4, 6],

"price": [2000000, 2500000, 3000000, 3500000, 4000000, 4500000, 5000000, 6000000] }

df = pd.DataFrame(data)

print(df)

X = df[["area", "length", "breadth", "rooms"]]

Y = df["price"]

Y = Y.values.reshape(-1, 1)

X_train, X_test, Y_train, Y_test = train_test_split(X, Y,

test_size=0.2,
random_state=42)

model = LinearRegression()

model.fit(X_train, Y_train)

Y_pred = model.predict(X_test)

print("Predicted Prices")

print(Y_pred)


```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
```

```
ruse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print(mae)
```

```
print(mse)
```

```
print(ruse)
```

```
print(r2)
```

```
area = float(input("Enter area"))
```

```
length = float(input("Enter length"))
```

```
breadth = float(input("Enter breadth"))
```

```
rooms = int(input("Enter no. of rooms"))
```

```
new_data = np.array([[area, length, breadth, rooms]])
```

```
pred_price = model.predict(new_data)
```

```
print(pred_price)
```

```
# output :- 2200
```

```
50
```

```
25
```

```
4
```

```
his :- 5697969 5431472
```

```
(ii) import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,
recall_score, precision_score, confusion_matrix.
```

```
data = {
    "area": [800, 900, 1200, 1500, 1700, 2000, 2200],
    "length": [20, 22, 25, 28, 30, 35, 38],
    "breadth": [40, 41, 42, 50, 52, 55, 58],
    "balk": [1, 2, 2, 4, 3, 5, 3],
    "size": [0, 0, 0, 1, 1, 1, 1]
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
X = df[['area', 'length', 'breadth', 'balk']]
```

```
Y = df['size']
```

```
y = Y.values.reshape(-1, 1)
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=
random_state=42)
```



```
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print(y_pred)
```

```
print(accuracy_score(y_test, y_pred))
```

```
area = float(input("Enter area"))
```

```
length = float(input("Enter length"))
```

```
breadth = float(input("Enter breadth"))
```

```
bhk = int(input("Enter bhk"))
```

```
new_data = [[area, length, breadth, bhk]]
```

```
prediction = model.predict(new_data)
```

```
if (prediction[0] == 1)
```

```
    print("big apartment")
```

```
else:
```

```
    print("small apartment")
```

Output:-

2200
45
66
8

— big apartment

Apply working of K-means.

```
→ import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
```

```
data = {
```

```
    "Hours-study": [2, 3, 5, 6, 8, 9, 1, 4, 7, 10],
```

```
    "Gpa": [1.0, 6.5, 7.5, 8.0, 9.0, 9.5, 5.0, 7.0, 8.5, 9.8],
```

```
    "Package": [5, 6, 7, 8, 9, 10, 4, 6, 8, 10],
```

```
    "Projects": [1, 1, 2, 2, 3, 3, 1, 2, 2, 3]
```

```
    "Marks": [50, 55, 65, 70, 85, 90, 40, 10, 75, 95]
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
X = df[['Hours-study', 'Gpa', 'Projects', 'Package']]
```

```
X_train, X_test = train_test_split(X, test_size=0.2,  
                                   random_state=42)
```

```
Scaler = StandardScaler()
```

```
scaled_X_train = Scaler.fit_transform(X_train)
```

```
scaled_X_test = Scaler.transform(X_test)
```


$Km = Kmeans(n-clusters = 3, random-state = 42)$

$Km.fit(scaled-X-train)$

$train_pred = Km.predict(scaled-X-train)$

$test_pred = Km.predict(scaled-X-test)$

$X-train['cluster'] = train_pred$

$X-test['cluster'] = test_pred$

$plt.scatter(X-train.iloc[:,0], X-train.iloc[:,1],
c=train_pred)$

$plt.title('K-means clustering')$

$plt.xlabel('Hours-study')$

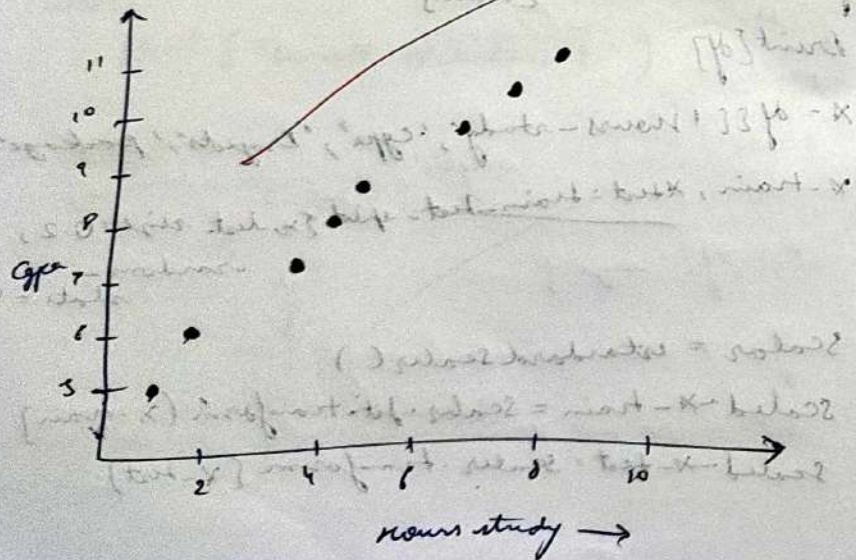
$plt.ylabel('Cgpa')$

$plt.show()$

from sklearn.metrics import silhouette_score

$SS = silhouette_score(scaled-X-train, train_pred)$

$print(SS)$



Q

⇒ Import numpy as np

Import pandas as pd

from sklearn.preprocessing import StandardScaler

from sklearn.tree import DecisionTreeRegressor

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score

data = {

"Hours-study": [2, 3, 5, 6, 8, 9, 1, 4, 7, 10],

"Cgpa": [6.0, 6.5, 7.5, 8.0, 9.0, 9.5, 5.0, 7.0, 8.5, 9.8],

"Portfolio": [5, 6, 7, 8, 9, 10, 4, 18, 10],

"Projects": [1, 1, 2, 2, 3, 3, 1, 2, 2, 3],

"marks": [50, 55, 65, 70, 80, 90, 10, 40, 75, 95]

}

$df = pd.DataFrame(data)$

$print(df)$

$X = df.iloc[:, :-1]$

$Y = df.iloc[:, -1]$

$X-train, X-test, y-train, y-test = train_test_split(X, Y, test_size=0.2, random_state=42)$

random-state=42

→

scaler = StandardScaler()

x_train_scaled = scaler.fit_transform(x_train)

x_test_scaled = scaler.transform(x_test)

model = LinearRegression()

model.fit(x_train_scaled, y_train)

y_pred = model.predict(x_test_scaled)

y_pred

print(r2_score(y_test, y_pred))

Output = 0.59375 or 59.375%

Abhinav

29-04-26